

CODLOCK

Backend Core

Engineering Status & Production-Readiness Report

AI-driven trust & confidence layer for social e-commerce (COD refusal mitigation) — Instagram & WhatsApp, Tunisia.

Prepared for the CODLOCK team

10 August 2026 · Branch: backend · NestJS Backend Core v0.1.0

Executive summary

STATUS: NOT PRODUCTION-READY — ONE OPERATOR STEP REMAINS

The NestJS Backend Core is the secure proxy between the seller-facing app and the Codlock Orchestrator Agent. It owns the order lifecycle, tenant isolation, risk-based deposits, virtual fitting, payment webhooks and seller analytics. The code, database SQL and test suites are complete and green. The single remaining blocker is a deployment step: the target Supabase database has no schema applied, so the real API-to-database path has not yet been exercised. Per our own rule, production-readiness is not claimed until that path is tested.

248 TOTAL TESTS	118 / 63 / 67 UNIT / E2E / DATABASE	6 OF 8 P1 ITEMS DONE	1 PRODUCTION BLOCKER
---------------------------------------	---	--	--

At a glance

Dimension	State
Architecture & layering	Verified
Database SQL correctness	Verified — real PostgreSQL 18
Security posture	Good — 3 findings fixed, 2 deferred by decision
Resilience	Verified — timeout, retry, circuit breaker
Test coverage	Strong — 248 tests, all green
CI / tooling	Verified — lint, typecheck, 3 suites, build
Database deployment	Blocked — operator step
Real API to database path	Not tested — gated on deployment

Architecture

The core never talks to Gravy or the AI agents directly. Every AI-facing call funnels through a single gateway (**OrchestratorService**) that applies one uniform resilience policy — timeout, bounded retry with backoff, and a circuit breaker. That single choke point is the strongest design decision in the codebase.



Data stores

Store	Role	Notes
Supabase (PostgreSQL)	System of record — catalog, orders, customers, fitting sessions, webhook ledger	Accessed via PostgREST with the service-role key. RLS on every table; no client policies.
Firebase Firestore	Append-only risk & analytics event log	Non-critical by design — fire-and-forget, degrades to no-ops if unconfigured.
PGLite (dev only)	Real PostgreSQL 18 for tests	Runs the shipped SQL in CI. Never ships to production.

Why deployment needs an operator: the backend reaches the database only through PostgREST, which does not execute DDL, and no direct Postgres connection string or Management API token exists in the repo. The schema must be applied once in the Supabase SQL editor.

Technology stack

Node 22 · TypeScript 5.7 · NestJS 10 (Express) · Supabase JS · firebase-admin · passport-jwt (HS256) · class-validator · Joi config validation · Helmet · @nestjs/throttler · Swagger/OpenAPI · Jest + supertest + PGLite · ESLint 9 / Prettier 3.

What the backend does

Order lifecycle (enforced state machine, illegal moves → HTTP 409)

DRAFT → PREVIEW_GENERATED → RISK_EVALUATED → DEPOSIT_PENDING → DEPOSIT_PAID → READY_TO_SHIP → SHIPPED → ACCEPTED | REFUSED. Pre-fulfilment states may also move to **CANCELLED** (new in this pass). Every transition uses optimistic locking (version + status guard) to prevent double-processing.

Modules

Module	Responsibility
Auth	Global JWT guard; token subject validated as a UUID (the tenant key).
Orchestrator	Single resilient gateway to risk / fitting / payment agents.
Orders	Lifecycle, catalog-authoritative pricing, cancellation, chat-webhook ingest.
Risk	0–100 score → tier → dynamic deposit; local fallback when the agent is down.
Fitting	Virtual try-on; validates customer/product/order ownership.
Payments	Gravv webhook, HMAC-verified, transactional dedup in PostgreSQL.
Customers / Products	Seller-scoped CRUD; risk history is not API-editable.
Analytics	Seller KPIs aggregated in SQL (correct past PostgREST row limits).

Cross-cutting

- Uniform response envelope { success, data, error, meta } on success and failure.
- Strict DTO validation; unknown properties rejected, not dropped.
- Tenant isolation on every query; cross-seller access returns 404.
- Rate limiting (verified 120/min); config seam ready for a shared Redis store.
- Fail-fast config validation at boot; Swagger gated off in production.

Testing & verification

Three layers, each proving something the others cannot. All 248 tests pass, alongside typecheck, lint and build on every CI run.

Suite	Tests	Engine	Proves
Unit	118	none (pure logic)	State machine, risk tiers, HMAC, circuit breaker, orchestrator retry/backoff.
End-to-end	63	real AppModule + in-memory DB double	Routing, auth, tenant isolation, full lifecycle, webhooks, error mapping.
Database integration	67	real PostgreSQL 18 (PGLite)	Schema, migrations, all 6 stored functions, grants, RLS, optimistic locking.

Verified against the live Supabase project

- URL reachable (HTTP 200); service-role key accepted.
- Database confirmed empty — clean deploy, nothing to preserve.
- Deployment procedure and a read-only verifier (npm run db:verify) prepared and tested.

Not yet verified

- The real NestJS → PostgREST → PostgreSQL write path (blocked on deployment).
- Firebase Firestore in cloud mode (runs against the emulator here; non-critical).
- Live Gravy and Orchestrator endpoints (exercised via stubs only).
- Behaviour behind more than one instance (throttler is in-memory).

Security findings

ID	Finding	Status
S-1	Webhook validation ran before HMAC check, leaking the DTO schema to anonymous callers	Fixed
S-3	No replay protection on the chat webhook	Fixed
E-1	Database errors collapsed to 500 (duplicate returned 500, not 409)	Fixed
B-1	requestDeposit could strand an order with a live, unmatchable payment	Fixed
B-2	Token subject trusted without validation	Fixed
M-1	Hardening migration unaplicable to a legacy DB holding data	Fixed
S-2	Single shared social webhook secret; sellerId caller-supplied	Deferred
S-4	Throttler in-memory — per-instance limits when scaled	Deferred

Strengths confirmed against a real engine: systematic tenant isolation, RLS on every table with no client policies, all stored functions locked to the service-role, constant-time HMAC verification, catalog-authoritative pricing, and risk aggregates that cannot be edited through the API.

P1 hardening — this delivery

#	Item	Status
1	Chat webhook replay protection (freshness window + event-id dedup)	Done
2	HMAC verification before DTO validation (guard)	Done
3	Per-seller webhook secrets	Deferred
4	PostgreSQL error mapping (23505→409, 23503→400)	Done
5	DEPOSIT_PENDING cancellation (manual endpoint + CANCELLED state)	Done
6	Redis-backed throttling	Seam only
7	Direct CircuitBreaker tests	Done
8	Direct Orchestrator retry/backoff tests	Done

Items 3 and 6 were deferred by explicit decision — per-seller secrets kept global for now, and throttling left as a config seam (no Redis dependency) until multi-instance deployment is required.

Production blockers

1

Deploy the database schema. Apply `db/schema.sql` in the Supabase SQL editor (procedure in `db/DEPLOYMENT.md`). The schema is proven to apply cleanly. Until this is done, every database-backed endpoint returns HTTP 500.

2

Run real API-to-database integration. Once the schema exists, exercise auth, tenant isolation, the full order lifecycle, risk, fitting, analytics and the Gravy webhook against a dedicated test seller/customer/order — the one path no test currently covers.

Neither is a code defect; both are gated on the operator deployment. The moment the schema is live, `npm run db:verify` confirms it and the integration pass can run.

Remaining work

Priority	Item
P1 (deferred)	Per-seller webhook secrets; Redis-backed throttling for multi-instance.
P2	Structured JSON logging; error tracking; Docker healthcheck; <code>.dockerignore</code> audit; graceful-shutdown tests.
Note	A PostgreSQL CI job is partly met already — CI runs real-PostgreSQL SQL tests via PGlite.

Reference documents in the repository

- `docs/PRODUCTION_READINESS.md` — full readiness detail behind this summary.
- `db/DEPLOYMENT.md` — exact schema deployment & verification procedure.
- `docs/HARDENING_PLAN.md` — P0–P2 plan mapped to files.
- `docs/BACKEND_REPORT.md` — architecture & module reference.

Verify this report: `npm run typecheck · lint:ci · test · test:e2e · test:db · build · db:verify`